
Le langage procédural PL/pgSQL

Dans tout l'énoncé, on suppose que le travail se fait avec le système d'information de la société Distribill (*i.e.* la base de données construite lors du TP2). Le schéma de la base de données est nommé **td4r206**.

Exercice 1 Définition d'une fonction PL/pgSQL

Première fonction PL/pgSQL

Proposez une fonction **anciennete** en PL/pgSQL qui détermine votre âge. Votre date de naissance est passée en argument à la fonction.

Il faudra penser au cas où la date d'anniversaire de l'année en cours est postérieure à la date courante. Il faut aussi que la date d'anniversaire passée en argument soit cohérente.

Usage :

```
select td4r206.anciennete('06/06/1984');
```

Pour vous inspirer, voici un exemple en SQL qui calcule le nombre d'années entre l'année courante et l'année 1984 :

```
select (extract(year from current_date)-extract(year from Date('06/06/1984')));
```

Application : Cette fonction pourrait vous servir à déterminer l'ancienneté dans l'entreprise arrondie à l'année des employés de **td4r206**.

Formulez la requête qui donne le matricule, le nom et l'ancienneté des employés.

Expression de requêtes dans une fonction PL/pgSQL

Exercice 2 On voudrait maintenant réaliser une fonction **nbdepts()** qui retourne un nombre entier. Y définir une variable entière **n**, puis lui affecter le nombre total de départements enregistrés dans la table **DEPT**. Retourner **n**. On pourra aller un peu plus loin et gérer le cas où aucun département n'est trouvé (déclencher une exception qui interrompt la fonction et affiche un message d'erreur au choix).

Application : Vous pourrez tester la fonction avec la requête suivante.

```
select td4r206.nbdepts();
```

Faites évoluer la requête de sorte à permettre de passer le nom du département en argument de la fonction. Dans ce cas, la fonction retournera le nombre de départements ayant le nom passé en argument.

Application : Vous pourrez tester la fonction avec la requête suivante.

```
select td4r206.nbdepts('Astronomie');
```

Exercice 3 Soit la table **td4r206._employe** qui stocke des informations sur les employés de Distribill. On souhaite déterminer la proportion d'ingénieurs parmi eux.

Écrire une fonction PL/pgSQL nommée **propIng()**, retournant un nombre réel et permettant de :

- Compter le nombre de n-uplets dans la table **td4r206._employe** et stocker le résultat dans une variable
- Compter le nombre d'employés dont la fonction est "INGENIEUR" et stocker le résultat dans une seconde variable

- Calculer la proportion (en pourcentage), stocker le résultat dans une troisième variable et retourner le résultat

Application : Vous pourrez tester la fonction avec la requête suivante.

```
select td4r206.propIng();
```

- On souhaite aller plus loin et inclure dans le programme précédent un traitement d'exception permettant à la fonction `propIng()` de renvoyer la valeur `NULL` si la table `td4r206._employe` est vide (ce qui provoque une division par zéro - Indice : exception système `division_by_zero`). Tester le bon fonctionnement de l'exception en suivant la procédure suivante :
 - Effacer le contenu de la table `td4r206._employe` (travailler sur une copie)
 - Exécuter la fonction `propIng()`

Exercice 4 On voudrait maintenant réaliser une fonction `nbsub` qui recherche le nombre de subordonnés d'un employé. On pourra réutiliser la requête récursive définie lors du TP précédent. Toutefois, on pourra aller un peu plus loin et gérer le cas où l'employé n'existe pas. Le matricule de l'employé est passé en argument de la fonction.

Application : Vous pourrez tester la fonction avec les requêtes suivantes.

```
select td4r206.nbsub(7698);
```

```
select td4r206.nbsub(1234);
```